

PSI Transition-State Pipeline Final Technical Report

Merged Detailed Flow, Cases, Equations, Algorithms, and Code-Level Audit

Project PSI

2026-07-07

Generated from the merged PSI pipeline technical report source. Equations are preserved as readable LaTeX-style notation so the PDF remains independent of external TeX/Pandoc installations.

Report Scope

This final report merges the two working technical documents:

- PSI_PIPELINE_DETAILED_PDF_READY.md: the main PDF-ready explanation with progression, cases, equations, and algorithms.
- PSI_PIPELINE_EXACT_FLOW.md: the code-level exact-flow audit.

The main body is organized for reading as a PDF. The appendix preserves the exact-flow audit trail so that implementation details remain traceable to the pipeline code.

Executive Summary

The PSI pipeline predicts a transition-state (TS) distance matrix, reconstructs TS coordinates, and predicts activation energy from reactant and product geometries. The production Ea prediction is the learned neural Ea head when the checkpoint contains that head. A Marcus/Hammond/OLS physics model is retained as a baseline and as a fallback for legacy checkpoints.

The pipeline has this progression:

Stage	Input	Main operation	Output
1. Parse	`.log` files or `extracted_dataset.json`	read coordinates and energies	raw entries
2. Group	raw entries	form `(reactant, product, TS)` triplets	reaction samples
3. Featurize	triplets	distances, masks, atom features, energy descriptors	tensors
4. Split	samples	deterministic stratified split	train/validation indices
5. Normalize	train samples	train-only z-score statistics	normalized features
6. Predict geometry	`D_R`, `D_I`, `D_P`	GRU/Transformer + geometry head + EGNN	`D_TS_pred`
7. Predict Ea	EGNN features + descriptors	attention pooled Ea head	`Ea_pred`
8. Train	predictions + targets	geometry, PINN, triangle, risk, Ea losses	checkpoints
9. Evaluate	best checkpoint	all-sample prediction + metrics	JSON/dashboard
10. Predict new case	new R/P logs	model + post-processing	JSON/XYZ

Important implementation fact:

- `psi_full_pipeline.py` and `psi_cloud_pipeline.py` share the same core model, but they are not identical.
- The cloud run under `runs/phase1_warm_start/train.log` used the cloud file, because it printed Using pre-extracted dataset.
- `psi_full_pipeline.py` currently includes coordinate noise and inverse-distance weighted geometry loss.
- `psi_cloud_pipeline.py` currently uses unweighted geometry loss and no coordinate noise.

Notation

Symbol	Meaning
---	---
N	maximum padded atom count, default $N=30$
n	real atom count for one reaction
X_R	reactant coordinates, shape $N \times 3$
X_P	product coordinates, shape $N \times 3$
X_{TS}	true TS coordinates, shape $N \times 3$
D_R	reactant pairwise distance matrix
D_P	product pairwise distance matrix
D_I	midpoint distance matrix, $(D_R + D_P)/2$
D_{TS}	true TS distance matrix
$\hat{D}_{TS}$	final predicted TS distance matrix
$\hat{D}_c$	coarse pre-EGNN predicted TS distance matrix
m_i	atom mask, 1 for a real atom and 0 for padding
M_{ij}	pair mask for valid non-diagonal atom pairs
E_R, E_P, E_{TS}	raw electronic energies in Hartree
e_R, e_P	endpoint energies in kcal/mol
E_a	activation energy target in kcal/mol
ΔE_{rxn}	signed reaction energy $e_P - e_R$
h_i	learned node feature for atom i
x_i	learned/refined 3D coordinate for atom i
r_i	covalent radius for atom i

Pair mask:

$$M_{ij} = m_i m_j (1 - \mathbf{1}_{i=j})$$

Distance matrix:

$$D_{ij}(X) = \sqrt{\sum_{k=1}^3 (X_{ik} - X_{jk})^2 + 10^{-8}}$$

Stage 1: Data Ingestion Cases

Case 1A: Full Local Pipeline Extraction

Applies to `psi_full_pipeline.py`.

Trigger:

```
python psi_full_pipeline.py train ...
```

Rule:

```
if extracted_dataset.json exists and force_extract is false:
    reuse extracted_dataset.json
else:
    open b97d3.tar.gz
    parse up to extraction_limit .log files
    save extracted_dataset.json
```

Default extraction-related configuration:

Key	Value
<code>`tar_path`</code>	<code>`b97d3.tar.gz`</code>
<code>`dataset_json`</code>	<code>`extracted_dataset.json`</code>
<code>`extraction_limit`</code>	<code>`60000`</code>
<code>`target_reactions`</code>	<code>`20000`</code>
<code>`force_extract`</code>	<code>`True`</code>

Output:

```
{
  "filename": "...",
  "energy": -123.456,
  "atoms": [
    {"atom": "C", "x": 0.0, "y": 0.0, "z": 0.0}
  ]
}
```

Case 1B: Cloud Pipeline Pre-Extracted Dataset

Applies to `psi_cloud_pipeline.py`.

Rule:

```
if extracted_dataset.json does not exist:
    raise FileNotFoundError
else:
    use extracted_dataset.json
```

This is the case used by the observed `phase1_warm_start` run.

Case 1C: Coordinate Parsing

The active parser in both Python files reads atom coordinates from Standard Nuclear Orientation blocks.

Parsing rule:

```
\text{atom} = \text{parts}[1],
\quad
x = \text{float(parts[2])},
\quad
y = \text{float(parts[3])},
\quad
z = \text{float(parts[4])}
```

Only rows with exactly five fields are accepted.

Important caveat:

PLANNING.md says that Coordinates (Angstroms) is handled, but the active code currently only matches Standard Nuclear Orientation for coordinates.

Case 1D: Energy Parsing

Energy is read from either of these lines:

Final energy is ...

Total energy in the final basis set = ...

The parser stores the last token:

`E = \text{float(last token)}`

Stage 2: Reaction Triplet Construction

Progression

Each raw entry is assigned to a reaction id and role:

```
rxn_id = second path segment
role = r if filename starts with r
role = p if filename starts with p
role = ts if filename starts with ts
```

A sample is built only when all three exist:

```
\{r,p,ts\} \subseq \text{roles}(rxn)
```

Case 2A: Accepted Reaction

A reaction is accepted if:

$$n \leq N$$

and, with negative-barrier filtering enabled:

$$E_a \geq 0$$

Default:

```
skip_negative_ea = True
max_atoms = 30
```

Case 2B: Rejected Reaction

A reaction is rejected if:

$$n > 30$$

or:

$$E_a < 0$$

when skip_negative_ea is true.

Activation Energy Equation

Raw energies are in Hartree and converted using:

$$C_H = 627.509 \text{ kcal mol}^{-1} \text{ Hartree}^{-1}$$

Endpoint energies:

```
e_R = C_H E_R,
\quad
e_P = C_H E_P
```

Activation energy target:

$$E_a = (E_{\text{TS}} - \max(E_R, E_P)) C_H$$

This means the barrier is relative to the higher-energy endpoint, not always the reactant.

Signed reaction energy:

$$\Delta E_{\text{rxn}} = e_P - e_R$$

Stage 3: Tensor and Feature Construction

3.1 Coordinate Padding

For each role $S \in \{R, P, TS\}$, coordinates are padded:

$$X_S \in \mathbb{R}^{N \times 3}$$

For real atoms:

$$x_{\{S,i\}} = [x_i, y_i, z_i]$$

For padding:

$$X_{\{S,i\}} = [0, 0, 0]$$

Mask:

```
m_i =
\begin{cases}
1, & \text{if } i < n \\
0, & \text{if } i \geq n
\end{cases}
```

3.2 Distance Matrices

The model is driven by invariant pairwise distances:

$$\begin{aligned} D_R &= D(X_R), \\ \quad\quad\quad &\text{\tiny\sf\scriptsize $\backslash$qqquad}\\ D_P &= D(X_P), \\ \quad\quad\quad &\text{\tiny\sf\scriptsize $\backslash$qqquad}\\ D_{TS} &= D(X_{TS}) \end{aligned}$$

The midpoint distance matrix is:

$$D_I = \frac{D_R + D_P}{2}$$

This is a distance-space midpoint, not necessarily a physically embedded coordinate midpoint.

3.3 Atom Vocabulary

Atom symbols are mapped to integer ids:

$$\text{id}(\text{atom}) \in \{1, \dots, K\}$$

Padding id:

$$\text{id(padding)} = 0$$

Observed run vocabulary:

```
{'C': 1, 'H': 2, 'N': 3, 'O': 4}
```

3.4 Per-Atom Physical Features

For atom \$i\$:

$$a_i = \begin{bmatrix} \chi_i, \\ z_i, \\ m_i \end{bmatrix}$$

where:

Term	Meaning
---	---

χ_i	electronegativity
Z_i	atomic number
M_i	atomic mass

Train-only normalization:

$$\tilde{a}_{i,d} = \frac{a_{i,d} - \mu_d}{\sigma_d}$$

with:

$$\sigma_d = \begin{cases} 1, & \sigma_d < 10^{-6} \\ \sigma_d, & \text{otherwise} \end{cases}$$

3.5 Fragment Kabsch Alignment for Descriptors

The model distances do not need coordinate alignment, but the 20D global descriptor uses reactant coordinates aligned to product coordinates.

Bond criterion:

$$\text{bond}_{ij}(X) = \mathbf{1} \left[D_{ij}(X) \leq 1.45(r_i + r_j) \right]$$

Fragments are connected components of this bond graph.

Fragment selection:

```
if product has more fragments than reactant:
    use product fragments
else if reactant has more fragments than product:
    use reactant fragments
else:
    use reactant fragments
```

For a fragment F :

$$X_{R,F}^{\text{aligned}} = \arg\min_{Q,t} \|X_{R,F}Q + t - X_{P,F}\|_F^2$$

where Q is a rotation/reflection matrix from Kabsch alignment and t is the centering translation implied by the implementation.

Singleton fragments are set directly to product coordinates.

3.6 20D Energy Descriptor

The global energy descriptor is:

$$g \in \mathbb{R}^{20}$$

Let:

$$\Delta_i = \|X_{R,i}^{\text{aligned}} - X_{P,i}\|_2$$

The first ten entries are:

$$g_{1:10} = \begin{bmatrix} |e_R - e_P|, \\ e_P - e_R, \end{bmatrix}$$

```

\operatorname{mean}(\delta),
\operatorname{std}(\delta),
\max(\delta),
n,
n_C,
n_H,
n_N,
n_O
]

```

Bond angle for a triplet (i,j,k) :

```

\theta_{ijk}
=
\cos^{-1}
\left(
\frac{(X_i-X_j)\cdot(X_k-X_j)}
{\|X_i-X_j\|_2\|X_k-X_j\|_2}
\right)

```

Angles are measured in degrees.

The last ten entries are:

```

g_{11:20} =
[
\mu(\theta_R),
\sigma(\theta_R),
\min(\theta_R),
\max(\theta_R),
\mu(\theta_P),
\sigma(\theta_P),
\min(\theta_P),
\max(\theta_P),
\mu(|\theta_R-\theta_P|),
\max(|\theta_R-\theta_P|)
]

```

The angle-change terms use only common triplet keys. If no common triplet exists:

```

\mu(|\theta_R-\theta_P|) = 0,
\quad
\max(|\theta_R-\theta_P|) = 0

```

Train-only normalization:

```

\tilde{g}_d = \frac{g_d - \mu^g_d}{\sigma^g_d}

```

Stage 4: Geometry Masks and Risk Features

4.1 Fragment Geometry Mask

The true TS geometry defines fragments:

$$\text{bond}^{\text{TS}}_{ij} = \mathbf{1}_{\left[D_{\text{TS},ij} \leq 1.45(r_i + r_j) \right]}$$

The geometry mask is:

$$G_{ij} = \begin{cases} 1, & \text{if } i \neq j \text{ and atoms } i, j \text{ are in the same TS fragment} \\ 0, & \text{otherwise} \end{cases}$$

This mask is used for masked geometry metrics and triangle loss.

4.2 Bond Sets

Reactant and product bond sets:

$$B_R = \{(i, j) : D_{R,ij} \leq 1.45(r_i + r_j)\}$$

$$B_P = \{(i, j) : D_{P,ij} \leq 1.45(r_i + r_j)\}$$

Formed and broken bonds:

$$B_{\text{formed}} = B_P \setminus B_R$$

$$B_{\text{broken}} = B_R \setminus B_P$$

Counts:

$$\begin{aligned} n_f &= |B_{\text{formed}}|, \\ n_b &= |B_{\text{broken}}|, \\ n_c &= n_f + n_b \end{aligned}$$

4.3 Active and Risky Pairs

Active pair:

$$A_{ij} = \mathbf{1}_{\left[|D_{R,ij} - D_{P,ij}| > 0.15 \right]}$$

Risky chemical bond types:

N-N, N-O, O-O, C-N, H-N

Risk pair mask:

$$R^{\text{pair}}_{ij} = \mathbf{1}_{\left[A_{ij}=1 \text{ or } \right]}$$

$$(i,j) \in B_{\text{formed}} \cup B_{\text{broken}}$$

Complexity flag:

$$c_{\text{flag}} = \mathbb{1}_{[n_c \geq 4 \text{ or } n_b \geq 3]}$$

Risky chemistry flag:

$$r_{\text{flag}} = \mathbb{1}_{[\text{any risky type appears in active/formed/broken pairs}]}$$

Risk score:

$$s_{\text{risk}} = c_{\text{flag}} + r_{\text{flag}}$$

4.4 Risk Penalty Cases

Runtime risk penalty is mode-dependent.

Case 4A: Margin Risk Penalty

This is the default.

$$p_{\text{risk}} = \max(0, n_c - 3)^2 + \max(0, 1 - n_c)^2 + 0.5 \mathbb{1}_{[r_{\text{flag}} > 0]}$$

Case 4B: Binary Risk Penalty

$$p_{\text{risk}} = \mathbb{1}_{[n_c \geq 4 \text{ or } n_b \geq 3]} + 0.5 \mathbb{1}_{[r_{\text{flag}} > 0]}$$

Case 4C: Sigmoid Risk Penalty

$$p_{\text{risk}} = \frac{1}{1 + \exp[-1.25(n_c - 4)]} + 0.5 \mathbb{1}_{[r_{\text{flag}} > 0]}$$

Stage 5: Split, Leakage Control, and Normalization

5.1 Split Progression

Default:

```
val_split = 0.1
split_seed = 42
split_strategy = stratified
split_bins = 5
```

Validation count:

```
n_{val}
=
\min
\left(
\max(1, \operatorname{round}(n_{\text{total}}v)),
n_{\text{total}}-1
\right)
```

where v is `val_split`.

5.2 Case 5A: Random Split

```
shuffle all indices with seed
validation = first n_val
train = remaining
```

5.3 Case 5B: Stratified Split

This is the default and the observed run behavior.

Energy quantile edges:

```
q_k = Q\left(\frac{k}{B}\right),
\quad
k=1, \ldots, B-1
```

with $B = \text{split_bins}$.

Stratum features:

```
b_E = \operatorname{searchsorted}(q, E_a)
b_N = \min(\lfloor n/5 \rfloor, 6)
b_C = \min(n_f + n_b, 4)
b_R = \mathbf{1}[s_{\text{risk}} > 0]
```

Stratum key:

```
K = (b_E, b_N, b_C, b_R)
```

Each stratum is shuffled with the same RNG seed. Validation examples are drawn from each stratum, then adjusted to match the exact global validation count.

5.4 Leakage Checks

The split must satisfy:

```
I_{\text{train}} \cap I_{\text{val}} = \emptyset
I_{\text{train}} \cup I_{\text{val}} = \{0, \ldots, n_{\text{total}}-1\}
```

No reaction id can appear in both train and validation:

```
\{\text{rxn\_id}(i): i \in I_{\text{train}}\}
```

```
\cap
\{\text{rxn\_id}\}(j):j\in I_{\text{val}}\}
=
\emptyset
```

Observed split:

	Set	Count	Ea mean	Ea std	Max Ea	Risk fraction
	All	16292	42.02	25.76	209.97	81.4%
	Train	14663	42.02	25.73	209.97	81.4%
	Validation	1629	42.01	26.01	142.80	81.3%

5.5 Train-Only Normalization

For any scalar/vector feature z :

$$\mu_z = \frac{1}{|I_{\text{train}}|} \sum_{i \in I_{\text{train}}} z_i$$

$$\sigma_z = \sqrt{\frac{1}{|I_{\text{train}}|} \sum_{i \in I_{\text{train}}} (z_i - \mu_z)^2}$$

Normalized value:

$$\tilde{z} = \frac{z - \mu_z}{\sigma_z}$$

Std guard:

$$\sigma_z = \begin{cases} 1, & \sigma_z < 10^{-6} \\ \sigma_z, & \text{otherwise} \end{cases}$$

Ea target:

$$y_a = \frac{E_a - \mu_{E_a}}{\sigma_{E_a}}$$

Signed reaction energy input:

$$\widetilde{\Delta E}_{\text{rxn}} = \frac{\Delta E_{\text{rxn}} - \mu_{\Delta E}}{\sigma_{\Delta E}}$$

Observed training stats:

Ea mean = 42.02 kcal/mol
 Ea std = 25.73 kcal/mol
 Delta E mean = 35.53 kcal/mol
 Delta E std = 31.08 kcal/mol

Stage 6: Model Forward Progression

6.1 Whole Model Summary

The model maps:

$$(\mathbf{D}_R, \mathbf{D}_I, \mathbf{D}_P, \text{atom ids}, \tilde{\mathbf{a}}, \widetilde{\Delta E}_{\text{rxn}}, \tilde{\mathbf{g}}) \rightarrow (\hat{\mathbf{D}}_{\text{TS}}, \hat{\mathbf{D}}_c, \hat{\mathbf{E}}_a)$$

The forward progression is:

1. Embed each distance row with Gaussian radial basis functions.
2. Append atom embedding and physical descriptors.
3. Encode each atom's R -> I -> P distance-row sequence with a GRU.
4. Contextualize atoms with Transformer attention.
5. Predict a coarse TS distance matrix.
6. Convert the coarse matrix to 3D coordinates by MDS.
7. Refine coordinates with EGNN.
8. Convert refined coordinates back to distances.
9. Predict normalized Ea from refined EGNN node features.

6.2 Gaussian Distance Embedding

Centers:

$$c_k = \text{linspace}(0.4, 6.0, 32)_k$$

Width:

$$\sigma_g = \frac{6.0 - 0.4}{32 - 1} \cdot 0.5$$

Embedding:

$$\begin{aligned} \phi_k(\mathbf{D}_{ij}) &= \exp \left[-\frac{1}{2} \left(\frac{\mathbf{D}_{ij} - c_k}{\sigma_g} \right)^2 \right] \end{aligned}$$

For atom i in state S :

$$\Phi_i^S = [\phi(\mathbf{D}_{i1}^S), \phi(\mathbf{D}_{i2}^S), \dots, \phi(\mathbf{D}_{iN}^S)]$$

6.3 Atom Feature Concatenation

Learned atom embedding:

$$\mathbf{u}_i = \text{Embedding}(\text{atom id}_i)$$

Atom feature:

$$\mathbf{z}_i = [\mathbf{u}_i, \tilde{\mathbf{a}}_i]$$

Per-state input:

$$r_i^S = [\Phi_i^S, z_i]$$

Input projection:

$$p_i^S = \text{Dropout} \left(\text{GELU} \left(\text{LayerNorm} (W_p r_i^S + b_p) \right) \right)$$

6.4 GRU Reaction-Path Encoding

For each atom, the state sequence is:

$$[p_i^R, p_i^I, p_i^P]$$

A bidirectional GRU processes this length-3 sequence. The implementation keeps the middle output:

$$h_i^{\text{GRU}} = \text{BiGRU}([p_i^R, p_i^I, p_i^P])_{\text{middle}}$$

Projection:

$$h_i^0 = \text{LayerNorm} \left(W_{\text{gru}} h_i^{\text{GRU}} + b_{\text{gru}} \right)$$

6.5 Transformer Atom Context

For each Transformer layer:

$$y = x + \text{MHA}(\text{LN}(x))$$

$$x^+ = y + \text{FFN}(\text{LN}(y))$$

Feed-forward network:

$$\text{FFN}(q) = W_2 \text{Dropout} \left(\text{GELU}(W_1 q + b_1) \right) + b_2$$

After three layers:

$$h_i^{\text{core}} = \text{LayerNorm}(x_i)$$

6.6 Coarse Geometry Head

For pair (i,j) :

$$q_{ij} = [h_i^{\text{core}}, h_j^{\text{core}}, z_i, z_j, D_{\{R, ij\}}, D_{\{I, ij\}}, D_{\{P, ij\}}]$$

]

MLP output:

$$[o^{\{\alpha\}}_{ij}, o^{\{\delta\}}_{ij}] = \text{operatorname{MLP}}_{\{\text{geom}\}}(q_{ij})$$

Interpolation coefficient:

$$\alpha_{ij} = \sigma(o^{\{\alpha\}}_{ij})$$

Distance correction:

$$\delta_{ij} = \text{operatorname{clip}}(o^{\{\delta\}}_{ij}, -3, 3)$$

Base interpolation:

$$B_{ij} = \alpha_{ij} D_{R, ij} + (1 - \alpha_{ij}) D_{P, ij}$$

Coarse TS distance:

$$\hat{D}_{c, ij} = \max(B_{ij} + \delta_{ij}, 0)$$

Symmetry and masking:

$$\begin{aligned} \hat{D}_{c, ij} &\leftarrow \frac{\hat{D}_{c, ij} + \hat{D}_{c, ji}}{2} \\ \hat{D}_{c, ij} &\leftarrow \hat{D}_{c, ij} M_{ij} \end{aligned}$$

6.7 MDS Coordinate Seed

The coarse distance matrix is detached before MDS:

$$\hat{D}_{c, ij}^{\{\text{detach}\}}$$

Squared distance matrix:

$$S_{ij} = \hat{D}_{c, ij}^{\{\text{detach}\}}{}^2 M_{ij}$$

Masked row mean:

$$\bar{S}_i = \frac{1}{n} \sum_j S_{ij}$$

Grand mean:

$$\bar{S} = \frac{1}{n^2} \sum_{ij} S_{ij}$$

Double-centered matrix:

$$C_{ij} =$$

$$-\frac{1}{2} \left(S_{ij} - \bar{S}_{i\cdot} - \bar{S}_{j\cdot} + \bar{S} \right)$$

Eigen-decomposition:

$$C \mathbf{v}_k = \lambda_k \mathbf{v}_k$$

Top three coordinate columns:

$$\mathbf{x}_{i,k}^{\text{init}} = \mathbf{v}_{i,k} \sqrt{\max(\lambda_k, 0)}$$

6.8 EGNN Refinement

Initial node feature:

$$\mathbf{h}_i^{\text{init}} = \text{LayerNorm} \left(\text{GELU}(W_e \mathbf{z}_i + b_e) \right)$$

For EGNN layer ℓ :

$$\begin{aligned} \mathbf{r}_{ij}^{\ell} &= \mathbf{x}_i^{\ell} - \mathbf{x}_j^{\ell} \\ d_{ij}^{\ell} &= \|\mathbf{r}_{ij}^{\ell}\|_2 \end{aligned}$$

Message:

$$\mathbf{m}_{ij}^{\ell} = \text{edgeMLP} \left([\mathbf{h}_i^{\ell}, \mathbf{h}_j^{\ell}, d_{ij}^{\ell}] \right)$$

Coordinate scalar:

$$c_{ij}^{\ell} = \text{coordMLP}(\mathbf{m}_{ij}^{\ell})$$

Raw coordinate translation:

$$\mathbf{t}_{ij}^{\text{raw}} = \mathbf{r}_{ij}^{\ell} c_{ij}^{\ell}$$

Implementation detail:

The code clamps the translation vector norm to at most `egnn_coord_clamp = 2.0`.

Coordinate update:

$$\mathbf{x}_i^{\ell+1} = \mathbf{x}_i^{\ell} + \mathbf{t}_{ij}^{\text{raw}}$$

$$+ \frac{1}{\max(\deg_i, 1)} \sum_j t_{ij}$$

where:

$$\deg_i = \sum_j M_{ij}$$

Node update:

$$\bar{m}_i^{\ell} = \sum_j m_{ij}^{\ell}$$

$$h_i^{\ell+1} = h_i^{\ell} + \operatorname{nodeMLP}\left(h_i^{\ell}, \bar{m}_i^{\ell}\right)$$

After four EGNN layers:

$$\begin{aligned} h_i^{\text{TS}} &= h_i^4, \\ x_i^{\text{TS}} &= x_i^4 \end{aligned}$$

Final predicted distance:

$$\begin{aligned} & \hat{D}_{TS,ij} \\ &= \\ & \sqrt{\frac{|x_i^{TS} - x_j^{TS}|^2}{2} + 10^{-8}} \\ & M_{ij} \end{aligned}$$

6.9 Learned Ea Head

Mean pooling:

$$h_{\text{mean}} = \frac{\sum_i m_i h_i^{\text{TS}}}{\max(\sum_i m_i, 1)}$$

Attention logits:

$$\ell_i = \text{operatorname{attnMLP}}(h_i^{\text{TS}})$$

Padded atoms are forced to a large negative logit:

$$\ell_i = -10^4 \quad \text{if } m_i = 0$$

Attention weights:

$$a_i = \frac{\exp(\ell_i)}{\sum_j \exp(\ell_j)}$$

Attention pooled descriptor:

$$h_{\text{attn}} = \sum_i a_i m_i h_i^{\text{TS}}$$

Ea feature:

$$f_{\{Ea\}} = [h_{\{attn\}},$$

```

    h_{mean},
    \widetilde{\Delta E}_{rxn},
    \tilde{g}
]

```

Normalized Ea prediction:

```

\hat{y}_{Ea}
=
\operatorname{MLP}_{Ea}(f_{Ea})

```

Denormalized Ea:

```

\hat{E}_a
=
\hat{y}_{Ea}\sigma_{Ea}+\mu_{Ea}

```

Stage 7: Training Losses and Cases

7.1 Huber and SmoothL1 Definitions

Geometry uses PyTorch Huber loss with $\delta = 0.5$:

$$H_{\{0.5\}}(e) = \begin{cases} 0.5e^2, & |e| < 0.5 \\ 0.5(|e| - 0.25), & |e| \geq 0.5 \end{cases}$$

Ea uses PyTorch SmoothL1 loss with default $\beta = 1$:

$$S(e) = \begin{cases} 0.5e^2, & |e| < 1 \\ |e| - 0.5, & |e| \geq 1 \end{cases}$$

7.2 Case 7A: Cloud Geometry Loss

Cloud geometry loss is unweighted over valid non-diagonal pairs:

$$L_{\{geom\}} = \frac{\sum_{ij} H_{\{0.5\}}(\left(\hat{D}_{\{TS, ij\}} - D_{\{TS, ij\}}\right)M_{\{ij\}})}{\max(\sum_{ij} M_{\{ij\}}, 1)}$$

Coarse auxiliary geometry loss:

$$L_{\{coarse\}} = \frac{\sum_{ij} H_{\{0.5\}}(\left(\hat{D}_{\{c, ij\}} - D_{\{TS, ij\}}\right)M_{\{ij\}})}{\max(\sum_{ij} M_{\{ij\}}, 1)}$$

This is the geometry-loss form visible in `psi_cloud_pipeline.py`.

7.3 Case 7B: Full Pipeline Inverse-Distance Geometry Loss

The full pipeline adds inverse-distance weighting:

$$W_{\{ij\}} = \frac{1}{D_{\{TS, ij\}}M_{\{ij\}} + 1}$$

Weighted pair mask:

$$M^{\wedge}W_{\{ij\}} = M_{\{ij\}}W_{\{ij\}}$$

Implemented refined geometry loss:

$$L_{\{geom\}} = \frac{\sum_{ij} H_{\{0.5\}}(\left(\hat{D}_{\{TS, ij\}} - D_{\{TS, ij\}}\right)M_{\{ij\}})}{\max(\sum_{ij} M_{\{ij\}}, 1)}$$

$$\left(\hat{D}_{TS, ij} - D_{TS, ij} \right) M^W_{ij} \\ \right) \\ \{ \\ \max(\sum_{ij} M^W_{ij}, 1) \\ \}$$

Implemented coarse geometry loss:

$$L_{\text{coarse}} \\ = \\ \frac{1}{\sum_{ij} H_{0.5}} \\ \left(\hat{D}_{c, ij} - D_{TS, ij} \right) M^W_{ij} \\ \right) \\ \{ \\ \max(\sum_{ij} M^W_{ij}, 1) \\ \}$$

Interpretation:

Short chemical bonds receive larger relative emphasis than long inter-fragment distances.

7.4 Spectator PINN Loss

Spectator mask:

$$S_{ij} \\ = \\ \mathbb{1}[|D_{R, ij} - D_{P, ij}| < 0.15] M_{ij}$$

Loss:

$$L_{\text{spectator}} \\ = \\ \frac{1}{\sum_{ij} S_{ij}} \\ \left[\hat{D}_{TS, ij} - D_{I, ij} \right] S_{ij} \\ \right]^2 \\ \{ \\ \max(\sum_{ij} S_{ij}, 1) \\ \}$$

This is included as:

$$0.2 L_{\text{spectator}}$$

7.5 Triangle Inequality Loss

For distinct triplets (i, j, k) :

$$v_{ijk} \\ = \\ \max(0, D_{ij} - D_{ik} - D_{kj} - 0.02)$$

The code also requires the triplet to be valid within the geometry mask:

$$T_{ijk} \\ = \\ m_i m_j m_k G_{ij} G_{ik} G_{kj}$$

Triangle loss for a distance matrix D :

$$L_{\text{triangle}}(D) \\ =$$

$$\frac{\sum_{ijk} T_{ijk} v_{ijk}^2}{\max(\sum_{ijk} T_{ijk}, 1)}$$

Combined triangle loss:

$$L_{\text{triangle, total}} = 1.0 L_{\text{triangle}}(\hat{D}_{\text{TS}}) + 0.25 L_{\text{triangle}}(\hat{D}_{\text{c}})$$

Total contribution:

$$0.05 L_{\text{triangle, total}}$$

Default triplet sampling:

$$\text{triangle_triplet_samples} = 1024$$

7.6 Risk Geometry Loss

Risk scale:

$$\rho_b = \min(1 + 0.5 p_{\text{risk}, b}, 3.0)$$

Risk pair weight:

$$\Omega_{b, ij} = R^{\text{pair}}_{b, ij} M_{b, ij} \rho_b$$

Risk geometry loss:

$$L_{\text{risk, geom}} = \frac{\sum_{b, ij} H_{0.5}(\hat{D}_{\text{TS}, b, ij} - D_{\text{TS}, b, ij}) \Omega_{b, ij}}{\max(\sum_{b, ij} \Omega_{b, ij}, 1)}$$

Contribution:

$$0.2 L_{\text{risk, geom}}$$

7.7 Ea Tail Weight Cases

If tail weighting is disabled:

$$w_b^{\text{tail}} = 1$$

If enabled:

$$w_b^{\text{tail}} = \begin{cases} 1.0, & E_{a, b} < 80 \\ 1.5, & 80 \leq E_{a, b} < 100 \\ 2.0, & 100 \leq E_{a, b} < 120 \\ 2.5, & E_{a, b} \geq 120 \end{cases}$$

Then:

$$w_b^{\text{tail}} \rightarrow \min(w_b^{\text{tail}}, 2.5)$$

Implementation defaults:

File	Default tail weighting
`psi_full_pipeline.py`	enabled
`psi_cloud_pipeline.py`	disabled

7.8 Ea Loss

Normalized target:

$$y_{\{a,b\}} = \frac{E_{\{a,b\}} - \mu_{\{Ea\}}}{\sigma_{\{Ea\}}}$$

Prediction error:

$$e_{\{a,b\}} = \hat{y}_{\{a,b\}} - y_{\{a,b\}}$$

Per-sample loss:

$$\ell_{\{a,b\}} = S(e_{\{a,b\}})$$

Weighted Ea loss:

$$L_{\{Ea\}} = \frac{\sum_b w_b^{\text{tail}} \ell_{\{a,b\}}}{\max(\sum_b w_b^{\text{tail}}, 1)}$$

Human-readable Ea MAE:

$$\text{EaMAE} = \frac{1}{B} \sum_b |\hat{y}_{\{a,b\}} - y_{\{a,b\}}| \sigma_{\{Ea\}}$$

7.9 Ea Schedule Cases

Defaults:

```
ea_loss_start_epoch = 1
ea_warmup_epochs = 150
ea_warmup_loss_weight = 1.0
ea_loss_weight = 2.0
ea_detach_during_warmup = True
```

Case 7C: Before Ea Start

Condition:

$$\text{epoch} < \text{ea_loss_start_epoch}$$

Ea weight:

$$\lambda_{\{Ea\}} = 0$$

Effect:

Only geometry, spectator, triangle, and risk geometry losses train.

Case 7D: Ea Warmup

Condition:

```
ea\_loss\_start\_epoch
\leq
epoch
\leq
ea\_warmup\_epochs
```

Ea weight:

```
\lambda_{Ea}=1.0
```

If detach is enabled:

```
h_{Ea}=stopgrad(h^{TS})
```

Effect:

The Ea head learns, but Ea gradients do not reshape the EGNN/backbone.

Case 7E: Joint Ea Training

Condition:

```
epoch > ea\_warmup\_epochs
```

Ea weight:

```
\lambda_{Ea}=2.0
```

Feature path:

```
h_{Ea}=h^{TS}
```

Effect:

Ea gradients flow through the Ea head into the EGNN.

7.10 Risk Ea Loss

Only active during joint Ea training.

Sample risk indicator:

```
q_b=\mathbf{1}[p_{risk,b}>0]
```

Risk Ea loss:

```
L_{risk,Ea}
=
\frac{
\sum_b \ell_{a,b} \rho_b q_b
}{
\max(\sum_b \rho_b q_b, 1)
}
```

Contribution:

```
0.5L_{risk,Ea}
```

7.11 Total Loss

Base loss:

```
L_{base}
=
L_{geom}
+
```

$$\begin{aligned}
& 0.5L_{\{\text{coarse}\}} \\
& + \\
& 0.2L_{\{\text{spectator}\}} \\
& + \\
& 0.05L_{\{\triangle, \text{total}\}}
\end{aligned}$$

Full batch loss:

$$\begin{aligned}
& L \\
& = \\
& L_{\{\text{base}\}} \\
& + \\
& 0.2L_{\{\text{risk}, \text{geom}\}} \\
& + \\
& \lambda_{\{\text{Ea}\}}L_{\{\text{Ea}\}} \\
& + \\
& \mathbf{1}_{\{\text{joint}\}}0.5L_{\{\text{risk}, \text{Ea}\}}
\end{aligned}$$

Terms are included only when the corresponding predictions/risk samples exist.

Stage 8: Optimizer, Scheduler, and Checkpoint Selection

8.1 Optimizer

The optimizer is AdamW with two parameter groups.

Base model:

$$\text{lr}_{\{\text{base}\}} = 1.5 \times 10^{-4}$$

$$\text{wd}_{\{\text{base}\}} = 10^{-2}$$

Ea head:

$$\text{lr}_{\{\text{Ea}\}} = 3.0 \times 10^{-4}$$

$$\text{wd}_{\{\text{Ea}\}} = 10^{-3}$$

Gradient clipping:

$$\|\nabla\|_2 \rightarrow \min(\|\nabla\|_2, 1.0)$$

8.2 Cosine Scheduler With Warmup

The code uses CosineAnnealingWarmup.

Hard-coded scheduler warmup in train_pipeline:

```
warmup_epochs = 100
min_lr = 1e-6
```

Warmup phase:

$$\text{lr}(t) = \text{lr}_0 \frac{t+1}{100}$$

Cosine phase:

```
progress
=
\frac{t-100}{T-100}

c(t)
=
\frac{1}{2}
\left[
1 + \cos(\pi \text{ progress})
\right]

lr(t)
=
lr_{\{\text{min}\}}
+
(\text{lr}_0 - \text{lr}_{\{\text{min}\}})c(t)
```

8.3 Checkpoint Selection Cases

Before joint Ea:

$$\text{val}_{\{\text{select}\}} = \text{val}_{\{\text{geom}\}}$$

After joint Ea begins:

```
val_{\{\text{select}\}}
=
val_{\{\text{geom}\}}
+
0.5val_{\{\text{Ea}, \text{norm}\}}
```

At:

epoch=ea_warmup_epochs+1

the best value and patience counter are reset. This prevents geometry-only warmup epochs from dominating model selection after Ea becomes a joint objective.

Stage 9: Evaluation and Metrics

9.1 Evaluation Progression

After training:

1. Load psi_best.pt.
2. Predict D_pred and neural Ea_pred for every sample.
3. Compute geometry metrics directly from raw neural predicted distances.
4. Reconstruct predicted TS coordinates for the physics baseline.
5. Fit physics OLS on training predictions only.
6. Evaluate neural Ea and physics Ea on train/validation/all.
7. Save detailed_analysis.json.

9.2 Geometry Metric

Masked geometry MAE:

$$\begin{aligned} \text{DistMAE} &= \\ &\frac{\sum_{ij} |\hat{D}_{TS, ij} - D_{TS, ij}| G_{ij}}{\max(\sum_{ij} G_{ij}, 1)} \end{aligned}$$

All-pair distance MAE:

$$\begin{aligned} \text{DistMAE}_{\text{all}} &= \\ &\operatorname{mean}_{ij} |\hat{D}_{TS, ij} - D_{TS, ij}| \end{aligned}$$

Dashboard geometry improvement:

$$\begin{aligned} \text{ImprovePct} &= \\ &\left(1 - \frac{\text{MAE}(\hat{D}_{TS}, D_{TS})}{\text{MAE}(D_I, D_{TS})} \right) 100 \end{aligned}$$

9.3 Energy Metrics

Energy error:

$$\epsilon_i = \hat{E}_{a, i} - E_{a, i}$$

MAE:

$$\text{MAE} = \frac{1}{n} \sum_i |\epsilon_i|$$

RMSE:

$$\begin{aligned} \text{RMSE} &= \\ &\sqrt{\frac{1}{n} \sum_i \epsilon_i^2} \end{aligned}$$

R^2 :

$$\begin{aligned} R^2 &= \\ &1 - \frac{\sum_i (E_{a, i} - \hat{E}_{a, i})^2}{\sum_i (E_{a, i} - \bar{E}_{a, i})^2} \end{aligned}$$

$$\{\sum_i (E_{a,i} - \bar{E}_a)^2\}$$

Pearson correlation:

$$r = \frac{\sum_i (E_{a,i} - \bar{E}_a)(\hat{E}_{a,i} - \bar{\hat{E}}_a)}{\sqrt{\sum_i (E_{a,i} - \bar{E}_a)^2} \sqrt{\sum_i (\hat{E}_{a,i} - \bar{\hat{E}}_a)^2}}$$

MAPE:

$$\text{MAPE} = \frac{100}{\operatorname{mean}_i \frac{|E_{a,i} - \hat{E}_{a,i}|}{|E_{a,i}|}}$$

with near-zero true values skipped by using nan.

Stage 10: Physics Ea Baseline

10.1 Purpose

The physics Ea model is a baseline:

primary Ea = learned neural Ea if available
 fallback Ea = physics Ea if neural head metadata is absent

10.2 Reorganization Energy

The bond set is the union of reactant, TS, and product bonds:

$$B_{\text{union}} = B_R \cup B_{\text{TS}} \cup B_P$$

For each bond:

$$\lambda = \sum_{(i,j) \in B_{\text{union}}} \frac{1}{2} k_{ij} \left[(D_{\text{TS},ij} - D_{\text{R},ij})^2 + (D_{\text{P},ij} - D_{\text{TS},ij})^2 \right]$$

Known k_{ij} values come from a table. Unknown pair fallback:

$$k_{ij} = \frac{500}{\max(r_i + r_j, 0.5)^3}$$

10.3 Hammond Index

For active bonded pairs:

$$|D_{\text{R},ij} - D_{\text{P},ij}| > 0.15$$

Sums:

$$S_R = \sum |D_{\text{TS},ij} - D_{\text{R},ij}|$$

$$S_P = \sum |D_{\text{TS},ij} - D_{\text{P},ij}|$$

Hammond index:

$$\eta = \begin{cases} 0.5, & \text{if } S_R + S_P < 10^{-8} \text{ (or no active bonds)} \\ \frac{S_R}{S_R + S_P}, & \text{otherwise} \end{cases}$$

Interpretation:

Eta	Meaning
---	---
near 0	early, reactant-like TS
near 1	late, product-like TS

10.4 Marcus Activation Feature

If $\lambda > 10^{-6}$:

$$E_{\text{Marcus}} = \frac{\lambda}{4}$$

$$\left(1 + \frac{\Delta E_{\text{rxn}}}{\lambda}\right)^2$$

Else:

$$E_{\text{Marcus}} = 0.5 |\Delta E_{\text{rxn}}|$$

10.5 OLS Calibration

Feature vector:

$$\mathbf{x} = \begin{bmatrix} E_{\text{Marcus}}, \\ \eta, \\ \Delta E_{\text{rxn}}, \\ 1 \end{bmatrix}$$

Training target:

$$y = E_a$$

OLS fit:

$$\begin{aligned} \hat{\beta} &= \arg\min_{\beta} \sum \|\mathbf{X}\beta - \mathbf{y}\|_2^2 \end{aligned}$$

Prediction:

$$\begin{aligned} \hat{E}_{a, \text{physics}} &= \mathbf{x}^T \hat{\beta} \end{aligned}$$

Stage 11: Prediction-Time Cases

11.1 Input Validation

New prediction requires:

$n_R = n_P$

Atom type ordering must match exactly:

$\text{atoms}_R[i] = \text{atoms}_P[i]$

Atom count must fit:

$n \leq 30$

Every atom type must exist in the training vocabulary.

11.2 Prediction Progression

1. Parse reactant and product logs.
2. Build D_R , D_P , D_I .
3. Build normalized atom physical features.
4. Build normalized signed reaction energy.
5. Build normalized 20D energy descriptor.
6. Run the neural model.
7. Denormalize E_a .
8. Post-process the predicted distance matrix.
9. Recover 3D coordinates with MDS.
10. Write JSON and optional XYZ.

11.3 E_a Source Case

If checkpoint has learned E_a metadata:

$\hat{E}_a = \hat{y}_{E_a} \sigma_{E_a} + \mu_{E_a}$

and:

$E_a_{\text{source}} = \text{neural}$

If not:

$\hat{E}_a = \hat{E}_{a, \text{physics}}$

and:

$E_a_{\text{source}} = \text{physics}$ (no learned head in checkpoint)

11.4 Steric Collision Clamp

Minimum distance:

$d^{\min}_{ij}$
 $=$
 $0.75(r_i + r_j)$

Clamp:

$\hat{D}_{ij}$
 $\rightarrow$
 $\max(\hat{D}_{ij}, d^{\min}_{ij})$

11.5 Spectator Constraint Clamp

For spectator pairs:

$$|D_{\{R, ij\}} - D_{\{P, ij\}}| \leq 0.15$$

Reference:

$$d^{\text{ref}}_{ij} = \frac{D_{\{R, ij\}} + D_{\{P, ij\}}}{2}$$

Bounds:

$$lo = 0.95 d^{\text{ref}}_{ij}$$

$$hi = 1.05 d^{\text{ref}}_{ij}$$

Clamp:

$$\hat{D}_{ij} \leftarrow \text{clip}(\hat{D}_{ij}, lo, hi)$$

11.6 Triangle Post-Processing

For every triplet:

$$\text{shortcut} = \hat{D}_{ik} + \hat{D}_{kj}$$

If:

$$\hat{D}_{ij} - \text{shortcut} > 0.05$$

then:

$$\hat{D}_{ij} = \hat{D}_{ji} = \text{shortcut}$$

Stage 12: Implementation Difference Summary

```
| Topic | `psi_full_pipeline.py` | `psi_cloud_pipeline.py` |
|---|---|---|
| Dataset input | can extract from `b97d3.tar.gz` | requires `extracted_dataset.json` |
| `ea_tail_weighting_enabled` default | `True` | `False` |
| Coordinate noise | `coord_noise = 0.05` on train R/P coords | absent |
| Main geometry loss | inverse-distance weighted | unweighted |
| Risk feature backfill | `ensure_sample_risk_features` exists | assumes fields exist |
| Parser coordinate block | `Standard Nuclear Orientation` | `Standard Nuclear Orientation` |
```

Clear interpretation:

- Use `psi_full_pipeline.py` as the most complete local implementation.
- Use `psi_cloud_pipeline.py` to understand the observed `phase1_warm_start` training run.
- Do not claim the cloud run used inverse-distance weighting unless that file is updated and the run log confirms it.

Algorithm A: Sample Construction

Input:

raw_data entries from extracted_dataset.json

Output:

samples, atom_vocab, atom_types_map

Procedure:

build atom_vocab from all atoms

group entries by rxn_id

for rxn_id in sorted reactions:

require roles r, p, ts

n = number of TS atoms

if n > max_atoms:

continue

pad X_R, X_P, X_TS to N x 3

build atom_ids and mask

$E_a = (E_{TS} - \max(E_R, E_P)) * 627.509$

if skip_negative_ea and $E_a < 0$:

continue

$e_r = E_R * 627.509$

$e_p = E_P * 627.509$

$de_{rxn} = e_p - e_r$

align reactant fragments to product for descriptors

build 20D energy features

build atom physical features

$D_R = \text{distance_matrix}(X_R)$

$D_P = \text{distance_matrix}(X_P)$

$D_{TS} = \text{distance_matrix}(X_{TS})$

build risk features from D_R and D_P

build geom_mask from true TS fragments

append sample

Algorithm B: Model Forward Pass

Input:

D_R, D_I, D_P, mask, atom_ids, atom_phys, de_rxn, energy_feats

Output:

D_pred, D_coarse, Ea_pred_norm

Procedure:

```

atom_emb = Embedding(atom_ids)
h_core = PSICore(D_R, D_I, D_P, mask, atom_ids, atom_phys)

D_coarse = GeometryHead(
    h_core,
    atom_emb,
    atom_phys,
    D_R,
    D_I,
    D_P,
    mask
)

if EGNN enabled:
    node_feats = concat(atom_emb, atom_phys)
    x_init = MDS(detach(D_coarse), mask)
    h_ts, x_ts = EGNN(node_feats, x_init, mask)
    D_pred = pairwise_distance(x_ts)

    if de_rxn is provided:
        if detach_ea_features:
            h_ea = detach(h_ts)
        else:
            h_ea = h_ts

        Ea_pred_norm = EaHead(h_ea, mask, de_rxn, energy_feats)
    else:
        Ea_pred_norm = None
else:
    D_pred = D_coarse
    Ea_pred_norm = None

return D_pred, D_coarse, Ea_pred_norm

```

Algorithm C: Training Epoch

Input:

model, loader, optimizer, epoch, stats, config

Procedure:

```

ea_started = epoch >= ea_loss_start_epoch
ea_joint = epoch > ea_warmup_epochs

if ea_joint:
    ea_weight = ea_loss_weight
else if ea_started:
    ea_weight = ea_warmup_loss_weight
else:
    ea_weight = 0

detach_ea_features =
    ea_detach_during_warmup and ea_started and not ea_joint

for batch in loader:
    move batch to device

    D_pred, D_coarse, Ea_pred_norm =
        model(..., detach_ea_features)

    compute geometry loss
    compute coarse geometry loss
    compute spectator loss
    compute triangle loss
    compute risk geometry loss when risk pairs exist

    loss =
        L_geom
        + 0.5 * L_coarse
        + 0.2 * L_spectator
        + 0.05 * L_triangle
        + 0.2 * L_risk_geom

    if Ea_pred_norm exists:
        y = (Ea - ea_mean) / ea_std
        ea_abs = SmoothL1(Ea_pred_norm - y)
        L_Ea = weighted_mean(ea_abs, tail_weights)

        if ea_weight > 0:
            loss += ea_weight * L_Ea

        if ea_joint and risk samples exist:
            loss += 0.5 * L_risk_Ea

    if training:
        if loss is non-finite:
            skip optimizer step

        backward with AMP GradScaler when enabled
        unscale gradients
        clip gradient norm to 1.0
        optimizer.step()

```

Algorithm D: Final Evaluation

Input:

best model, all samples, train/validation split

Procedure:

```
for every sample:
    run neural model
    store D_pred
    if Ea head exists:
         $Ea_{neural} = Ea_{norm} * ea_{std} + ea_{mean}$ 

    compute dist_MAE using geom_mask

for every sample:
    symmetrize D_pred
    zero diagonal
    apply steric clamp
    recover predicted TS coordinates by MDS

fit PhysicsEaCalculator on training samples only

for every sample:
    Ea_physics = physics model from predicted TS coordinates
    Ea_pred = Ea_neural if available else Ea_physics
    save detailed record

write detailed_analysis.json
write psi_final.pt
write dashboard
```

Appendix A: Code-Level Exact-Flow Audit

This appendix folds in the earlier exact-flow document. It intentionally keeps some overlap with the main body because its purpose is auditability: it records the implementation-level equations, branch behavior, and file differences in a more direct code-reference style.

This document is the code-level technical flow for the PSI transition-state pipeline in this workspace. It is based on the implementation in `psi_full_pipeline.py` and `psi_cloud_pipeline.py`.

Important scope note:

- `psi_full_pipeline.py` is the complete local pipeline: extraction from a tar archive, training, evaluation, prediction, and dashboard generation.
- `psi_cloud_pipeline.py` is the cloud/training variant: it expects an existing `extracted_dataset.json` and skips tar extraction.
- The latest observed run log under `runs/phase1_warm_start/train.log` matches the cloud variant because it prints Using pre-extracted dataset.
- The two files are not perfectly identical. Exact differences are listed in Section 17.

All equations below use implementation names where possible.

1. Pipeline Summary

The pipeline predicts transition-state (TS) geometry and activation energy from reactant and product geometries.

High-level flow:

1. Read Gaussian/Q-Chem-style .log entries or a pre-extracted JSON dataset.
2. Group logs into complete reaction triplets: reactant r, product p, TS ts.
3. Build per-reaction tensors:
 - reactant distance matrix `D_R`
 - product distance matrix `D_P`
 - midpoint distance matrix `D_I`
 - true TS distance matrix `D_TS`
 - atom ids, atom physical descriptors, global energy descriptors
 - fragment geometry mask and reaction risk features
4. Split data into train/validation using deterministic stratified sampling.
5. Train a neural model:
 - `PSICore`: distance-row Gaussian embedding + GRU + Transformer
 - `GeometryHead`: coarse TS distance matrix
 - `MDS`: distance matrix to 3D coordinates
 - `EGNN`: E(n)-equivariant coordinate refinement
 - `EaHead`: learned activation-energy head
6. Evaluate:
 - primary Ea comes from the learned neural Ea head
 - physics Ea is reported as a baseline using Marcus/Hammond/OLS equations
7. Save:
 - `psi_best.pt`
 - `psi_latest.pt`
 - `psi_final.pt`
 - `training_history.json`
 - `detailed_analysis.json`

- psi_results_dashboard.html

2. Log Parsing and Raw Dataset Extraction

2.1 Parsed geometry block

The active parser in both pipeline files reads atom coordinates only from the latest Standard Nuclear Orientation block:

```
atom = parts[1]
x = float(parts[2])
y = float(parts[3])
z = float(parts[4])
```

The parser expects each atom row in that block to split into exactly 5 fields.

2.2 Parsed energy lines

The active parser reads the final scalar energy from either:

```
Final energy is ...
Total energy in the final basis set = ...
```

It takes the last whitespace-separated token as a float.

2.3 Full vs cloud extraction

psi_full_pipeline.py:

```
if extracted_dataset.json exists and force_extract is false:
    skip extraction
else:
    read b97d3.tar.gz and parse up to extraction_limit .log files
```

psi_cloud_pipeline.py:

```
if extracted_dataset.json does not exist:
    raise FileNotFoundError
else:
    use pre-extracted dataset
```

3. Reaction Triplet Construction

Each JSON entry is assigned a role from its filename:

```
role = "r" if basename starts with "r"
role = "p" if basename starts with "p"
role = "ts" if basename starts with "ts"
```

For each rxn_id, a sample is kept only if all roles exist:

```
{r, p, ts}
```

The reaction is skipped if:

```
n_atoms > max_atoms
```

or, when skip_negative_ea = True, if:

```
Ea_raw < 0
```

Default maximum atoms:

```
max_atoms = 30
```

4. Core Coordinates, Distances, and Energies

Let:

```
N = max_atoms = 30
n = real atom count
X_R, X_P, X_TS in R^(N x 3)
m_i = 1 for i < n else 0
```

Coordinates are zero-padded to length N.

4.1 Pairwise distance matrix

For any coordinate matrix X:

$$D_{ij} = \sqrt{\sum_k (X_{i,k} - X_{j,k})^2 + 1e-8}$$

The pipeline builds:

```
D_R = distance_matrix(X_R)
D_P = distance_matrix(X_P)
D_I = (D_R + D_P) / 2
D_TS = distance_matrix(X_TS)
```

4.2 Activation energy target

Raw log energies are in Hartree. The conversion constant is:

$$\text{hartree_to_kcal} = 627.509$$

The target activation energy is:

$$E_{a_raw} = (E_{TS} - \max(E_R, E_P)) * 627.509$$

This means the barrier is measured relative to the higher-energy endpoint.

4.3 Signed reaction energy

Reactant and product energies are first converted to kcal/mol:

$$e_r = E_R * 627.509$$

$$e_p = E_P * 627.509$$

Signed reaction energy:

$$de_{rxn_raw} = e_p - e_r$$

This is the Bell-Evans-Polanyi driver used by the Ea head.

5. Atom and Reaction Features

5.1 Atom ids

The atom vocabulary is built from all parsed atom symbols:

```
atom_vocab = {atom_symbol: 1-based integer id}
padding_id = 0
```

The observed phase1_warm_start run used:

```
{'C': 1, 'H': 2, 'N': 3, 'O': 4}
```

5.2 Per-atom physical descriptors

For each real atom:

```
atom_phys_i = [electronegativity(atom_i), atomic_number(atom_i), atomic_mass(atom_i)]
```

Padding rows are zeros.

Training-split normalization:

$$\text{atom_phys_norm} = (\text{atom_phys_raw} - \text{aphys_mean}) / \text{aphys_std}$$

Any std smaller than 1e-6 is replaced by 1.0.

5.3 Reactant/product alignment for descriptors

For feature construction only, the reactant is fragment-aligned to the product.

Bond criterion:

$$\text{bond}(i,j) = D_{ij} \leq \text{fragment_bond_scale} * (r_{\text{cov}_i} + r_{\text{cov}_j})$$

$$\text{fragment_bond_scale} = 1.45$$

The fragment set is selected from whichever of reactant/product has more fragments; ties use reactant

fragments.

For each fragment:

```
if fragment size >= 2:
    align X_R_fragment to X_P_fragment by Kabsch
else:
    set singleton reactant coordinate to product coordinate
```

5.4 20D global energy feature vector

build_energy_features(...) returns:

```
[
    abs(e_r - e_p),
    e_p - e_r,
    mean_i ||X_R_aligned_i - X_P_i||,
    std_i ||X_R_aligned_i - X_P_i||,
    max_i ||X_R_aligned_i - X_P_i||,
    n,
    count_C,
    count_H,
    count_N,
    count_O,
    mean(angle_R),
    std(angle_R),
    min(angle_R),
    max(angle_R),
    mean(angle_P),
    std(angle_P),
    min(angle_P),
    max(angle_P),
    mean_common_triplets |angle_R - angle_P|,
    max_common_triplets |angle_R - angle_P|
]
```

Bond angles are computed from bonded triplets (i, j, k) with central atom j.

For vectors:

```
v1 = X_i - X_j
v2 = X_k - X_j
angle(i,j,k) = arccos( dot(v1,v2) / (||v1|| ||v2||) ) in degrees
```

Training-split normalization:

```
energy_feats_norm = (energy_feats_raw - efeat_mean) / efeat_std
```

Any std smaller than 1e-6 is replaced by 1.0.

6. Fragment Geometry Mask

Fragments are built from the true TS geometry using the same covalent-radius bond cutoff:

```
bond_TS(i,j) = D_TS_ij <= 1.45 * (r_cov_i + r_cov_j)
```

geom_mask_ij = 1 when atoms i and j are in the same TS fragment, else 0.

The diagonal is always zero:

```
geom_mask_ii = 0
```

This mask is used for geometry evaluation and for the triangle-inequality loss.

7. Risk Features

Bond sets:

```
B_R = {(i,j): D_R_ij <= 1.45 * (r_cov_i + r_cov_j)}
B_P = {(i,j): D_P_ij <= 1.45 * (r_cov_i + r_cov_j)}
```

```

formed = B_P - B_R
broken = B_R - B_P
formed_n = |formed|
broken_n = |broken|
changed_n = formed_n + broken_n

```

Risky bond types:

```
RISK_BOND_TYPES = {N-N, N-O, O-O, C-N, H-N}
```

Active pair:

```

active(i,j) = abs(D_R_ij - D_P_ij) > spectator_threshold
spectator_threshold = 0.15

```

Risk pair mask:

```
risk_pair_mask_ij = 1 if active(i,j) or (i,j) in formed or (i,j) in broken
```

Complexity and chemistry flags:

```

complexity_flag = 1 if changed_n >= 4 or broken_n >= 3 else 0
risky_chem_flag = 1 if any risky bond type appears in active/formed/broken pairs else 0
risk_score = complexity_flag + risky_chem_flag

```

Build-time margin penalty:

```

risk_penalty =
    max(0, changed_n - 3)^2
    + max(0, 1 - changed_n)^2
    + 0.5 * risky_chem_flag

```

Runtime continuous_risk_penalty depends on risk_penalty_mode. Defaults:

```

risk_penalty_mode = "margin"
risk_safe_min = 1.0
risk_safe_max = 3.0
risk_sigmoid_center = 4.0
risk_sigmoid_k = 1.25

```

Modes:

binary:

```
complexity = 1 if changed_n >= safe_max + 1 or broken_n >= safe_max else 0
```

sigmoid:

```
complexity = 1 / (1 + exp(-sigmoid_k * (changed_n - sigmoid_center)))
```

margin:

```
complexity = max(0, changed_n - safe_max)^2 + max(0, safe_min - changed_n)^2
```

```
runtime_risk_penalty = complexity + 0.5 * I(risky_chem_flag > 0)
```

8. Train/Validation Split

Default split configuration:

```

val_split = 0.1
split_seed = 42
split_strategy = "stratified"
split_bins = 5

```

Total validation count:

```
n_val = min(max(1, round(n_total * val_split)), n_total - 1)
```

For stratified splitting:

```
ea_edges = unique(quantile(Ea_raw, [1/split_bins, ..., (split_bins-1)/split_bins]))
```

```

ea_bin = searchsorted(ea_edges, Ea_raw, side="right")
atom_bin = min(floor(n_atoms / 5), 6)
changed_bin = min(formed_n + broken_n, 4)
risk_bin = I(risk_score > 0)
stratum_key = (ea_bin, atom_bin, changed_bin, risk_bin)

```

For each stratum:

```

shuffle group with rng(seed)
group_val = round(len(group) * val_split)
if len(group) <= 1: group_val = 0
else: group_val = min(group_val, len(group) - 1)

```

After all strata, the code adjusts the validation set to exactly `n_val` by moving samples between train and validation.

Integrity checks:

```

train_indices intersect val_indices must be empty
no duplicate indices in either split
train union val must cover all samples
same rxn_id must not appear in both train and validation

```

The observed `phase1_warm_start` split:

```

total = 16292
train = 14663
validation = 1629
leakage = false

```

9. Normalization

All normalization statistics are computed on train indices only.

Activation energy:

```

ea_mean = mean_train(Ea_raw)
ea_std = std_train(Ea_raw)
if ea_std < 1e-6: ea_std = 1.0
Ea_target_norm = (Ea_raw - ea_mean) / ea_std

```

Signed reaction energy:

```

de_rxn_mean = mean_train(de_rxn_raw)
de_rxn_std = std_train(de_rxn_raw)
if de_rxn_std < 1e-6: de_rxn_std = 1.0
de_rxn_norm = (de_rxn_raw - de_rxn_mean) / de_rxn_std

```

Atom physical features and 20D energy features are also z-scored by train-only means and stds.

Observed `phase1_warm_start` stats:

```

Ea mean = 42.02 kcal/mol
Ea std  = 25.73 kcal/mol
de_rxn mean = 35.53 kcal/mol
de_rxn std  = 31.08 kcal/mol
model parameters = 3,199,272

```

10. Neural Architecture

Default architecture parameters:

```

n_gaussians = 32
gauss_start = 0.4
gauss_stop = 6.0
atom_embed_dim = 32
gru_hidden = 128
gru_layers = 2
gru_dropout = 0.3

```

```

attn_heads = 8
attn_layers = 3
ff_dim = 512
dropout = 0.35
delta_clamp = 3.0
egnn_enabled = True
egnn_layers = 4
egnn_hidden = 128
egnn_coord_clamp = 2.0
ea_head_dropout = 0.15

```

10.1 Gaussian distance embedding

Centers:

```

c_k = linspace(0.4, 6.0, 32)
sigma = (6.0 - 0.4) / (32 - 1) * 0.5

```

For each distance:

```
G_k(D_ij) = exp(-0.5 * ((D_ij - c_k) / sigma)^2)
```

For each atom i , the complete row embedding of $D[i, :]$ is flattened.

10.2 PSI core

For each atom:

```
z_i = concat(atom_embedding_i, atom_phys_norm_i)
```

For each geometry state S in $\{R, I, P\}$:

```

u_i^S = concat(flatten(G(D_S[i, :])), z_i)
e_i^S = input_proj(u_i^S)

```

input_proj:

```

Linear(N * n_gaussians + atom_embed_dim + 3, 256)
LayerNorm(256)
GELU
Dropout(0.35)

```

For each atom, the sequence $[e_i^R, e_i^I, e_i^P]$ is passed through a bidirectional GRU:

```

GRU input size = 256
GRU hidden per direction = 128
GRU output width = 256

```

The model takes the middle output:

```
context_i = GRU([R, I, P]) output at I
```

Then:

```
context_i = LayerNorm(Linear(context_i))
```

A stack of 3 pre-norm Transformer encoder layers processes atoms globally:

```

x2 = LayerNorm(x)
x = x + MultiHeadAttention(x2, x2, x2)
x2 = LayerNorm(x)
x = x + FeedForward(x2)

```

FeedForward:

```

Linear(256, 512)
GELU
Dropout
Linear(512, 256)
Dropout

```

Output:

`h_core_i` = final LayerNorm output

10.3 Geometry head

For each atom pair (i,j):

```
q_ij = concat(
    h_core_i,
    h_core_j,
    z_i,
    z_j,
    D_R_ij,
    D_I_ij,
    D_P_ij
)
```

The pair MLP:

```
Linear(pair_dim, 256)
GELU
Dropout
Linear(256, 128)
GELU
Dropout
Linear(128, 2)
```

The final linear layer is zero-initialized.

The two outputs are:

```
alpha_ij = sigmoid(out_ij, 0)
delta_ij = clamp(out_ij, 1, -3.0, 3.0)
```

Coarse TS distance:

```
D_base_ij = alpha_ij * D_R_ij + (1 - alpha_ij) * D_P_ij
D_coarse_ij = max(D_base_ij + delta_ij, 0)
```

The matrix is symmetrized and masked:

```
D_coarse = (D_coarse + D_coarse^T) / 2
D_coarse_ii = 0
D_coarse_ij = 0 for padded pairs
```

10.4 Classical MDS seed

The EGNN needs coordinates, so the coarse distance matrix is embedded by classical MDS. In the forward pass this MDS seed is detached from the graph.

For each molecule:

```
S = D_coarse^2
S is zeroed for padded pairs
row_mean_i = sum_j S_ij / n
grand = sum_ij S_ij / n^2
B_ij = -0.5 * (S_ij - row_mean_i - row_mean_j + grand)
```

The code symmetrizes B, shifts dummy padded diagonal modes downward, adds a small valid-block jitter, and takes the top 3 eigenpairs:

```
B v_k = lambda_k v_k
x_init_i,k = v_i,k * sqrt(max(lambda_k, 0))
```

Padded atom coordinates are zeroed.

10.5 EGNN coordinate refinement

Node input:

```
node_feats_i = concat(atom_embedding_i, atom_phys_norm_i)
h_i = embed_in(node_feats_i)
x_i = x_init_i
```

embed_in:

```
Linear(atom_embed_dim + 3, 128)
GELU
LayerNorm(128)
```

Each EGCL layer:

```
rel_ij = x_i - x_j
dist2_ij = ||rel_ij||^2
m_ij = edge_mlp(concat(h_i, h_j, dist2_ij)) * valid_pair_mask_ij
```

Coordinate update:

```
raw_trans_ij = rel_ij * coord_mlp(m_ij)
trans_ij = raw_trans_ij with vector norm clamped to <= 2.0
x_i = x_i + sum_j trans_ij / degree_i
```

Node update:

```
agg_i = sum_j m_ij
h_i = h_i + node_mlp(concat(h_i, agg_i))
```

After 4 EGCL layers:

```
h_ts_i = refined node feature
x_ts_i = refined coordinate
```

Refined predicted TS distances:

```
D_pred_ij = sqrt(||x_ts_i - x_ts_j||^2 + 1e-8)
```

The diagonal and padded pairs are zeroed.

10.6 Learned Ea head

The Ea head consumes refined EGNN node features.

Masked mean pooling:

```
h_mean = sum_i m_i * h_ts_i / sum_i m_i
```

Attention pooling:

```
logit_i = attn_mlp(h_ts_i)
logit_i = -1e4 for padded atoms
a_i = softmax(logit_i over atoms)
h_attn = sum_i a_i * h_ts_i * m_i
```

Ea feature vector:

```
f_ea = concat(h_attn, h_mean, de_rxn_norm, energy_feats_norm)
```

MLP:

```
Linear(2 * 128 + 1 + 20, 128)
GELU
Dropout(0.15)
Linear(128, 64)
GELU
Dropout(0.15)
Linear(64, 1)
```

The final linear layer is initialized with Xavier uniform gain 0.1 and zero bias.

Output:

```
Ea_pred_norm = EaHead(f_ea)
Ea_pred_kcal = Ea_pred_norm * ea_std + ea_mean
```

11. Training Losses

11.1 Common masks

For a batch:

```
M_ij = m_i * m_j * (1 - I_ij)
```

where I_{ij} is the identity matrix.

11.2 Huber and SmoothL1 definitions

$F.\text{huber_loss}(\dots, \text{delta}=0.5)$:

```
Huber_0.5(e) =
    0.5 * e^2           if |e| < 0.5
    0.5 * (|e| - 0.25)  otherwise
```

$F.\text{smooth_l1_loss}(\dots)$ with default beta 1.0:

```
SmoothL1(e) =
    0.5 * e^2           if |e| < 1
    |e| - 0.5           otherwise
```

11.3 Main geometry loss

There are two implemented variants.

Cloud pipeline geometry loss:

```
denom = sum_ij M_ij
L_geom = sum_ij Huber_0.5((D_pred_ij - D_TS_ij) * M_ij) / denom
L_geom_coarse = sum_ij Huber_0.5((D_coarse_ij - D_TS_ij) * M_ij) / denom
```

Full pipeline geometry loss:

```
W_ij = 1 / (D_TS_ij * M_ij + 1)
MW_ij = M_ij * W_ij
denom_weighted = sum_ij MW_ij
L_geom = sum_ij Huber_0.5((D_pred_ij - D_TS_ij) * MW_ij) / denom_weighted
L_geom_coarse = sum_ij Huber_0.5((D_coarse_ij - D_TS_ij) * MW_ij) / denom_weighted
```

Note: in the full pipeline the weight is applied to both prediction and target before the Huber loss, so the implemented error is $MW_{ij} * (\text{prediction} - \text{target})$.

11.4 Spectator PINN loss

Spectator mask:

```
S_ij = I(abs(D_R_ij - D_P_ij) < spectator_threshold) * M_ij
spectator_threshold = 0.15
```

Loss:

```
L_pinn = sum_ij ((D_pred_ij - D_I_ij) * S_ij)^2 / max(sum_ij S_ij, 1)
```

This pulls non-changing distances toward the reactant/product midpoint.

11.5 Triangle inequality loss

Distinct triplets (i,j,k) are either all enumerated or sampled.

Default:

```
triangle_triplet_samples = 1024
triangle_tolerance = 0.02
```

For a predicted distance matrix D:

```
valid_ijk = m_i * m_j * m_k * geom_mask_ij * geom_mask_ik * geom_mask_kj
violation_ijk = max(0, D_ij - (D_ik + D_kj) - 0.02)
```

```
L_triangle(D) = sum_ijk valid_ijk * violation_ijk^2 / max(sum_ijk valid_ijk, 1)
```

Combined triangle loss:

```
L_triangle =
    triangle_refined_weight * L_triangle(D_pred)
    + triangle_coarse_weight * L_triangle(D_coarse)
```

```
triangle_refined_weight = 1.0
```

```
triangle_coarse_weight = 0.25
```

11.6 Risk geometry loss

Risk scale per sample:

```
risk_scale = clamp(1 + risk_weight_alpha * runtime_risk_penalty,
                    max = risk_weight_max)
```

```
risk_weight_alpha = 0.5
```

```
risk_weight_max = 3.0
```

Risk pair weight:

```
R_ij = risk_pair_mask_ij * M_ij * risk_scale
```

If any risk pair exists:

```
L_risk_geom =
    sum_ij Huber_0.5(D_pred_ij - D_TS_ij) * R_ij / max(sum_ij R_ij, 1)
```

Added with:

```
risk_geom_loss_weight = 0.2
```

11.7 Learned Ea loss

Normalized target:

```
y_ea = (Ea_raw - ea_mean) / ea_std
```

```
e_ea = Ea_pred_norm - y_ea
```

```
ea_abs = SmoothL1(e_ea)
```

Tail sample weights:

```
if ea_tail_weighting_enabled is false:
```

```
    w_tail = 1
```

```
else:
```

```
    bins = [80, 100, 120]
```

```
    values = [1.0, 1.5, 2.0, 2.5]
```

```
    w_tail = 1.0 if Ea < 80
```

```
    w_tail = 1.5 if Ea >= 80
```

```
    w_tail = 2.0 if Ea >= 100
```

```
    w_tail = 2.5 if Ea >= 120
```

```
    w_tail = min(w_tail, ea_tail_weight_max)
```

Weighted Ea loss:

```
L_ea_weighted = sum_b w_tail_b * ea_abs_b / max(sum_b w_tail_b, 1)
```

Unweighted Ea tracking loss:

```
L_ea = mean_b ea_abs_b
```

11.8 Ea warm-start schedule

Defaults:

```
ea_loss_start_epoch = 1
```

```
ea_warmup_epochs = 150
```

```
ea_warmup_loss_weight = 1.0
```

```
ea_loss_weight = 2.0
```

```
ea_detach_during_warmup = True
```

Per epoch:

```
ea_started = epoch >= ea_loss_start_epoch
ea_joint = epoch > ea_warmup_epochs
```

```
if ea_joint:
    ea_w = ea_loss_weight
elif ea_started:
    ea_w = ea_warmup_loss_weight
else:
    ea_w = 0
```

```
detach_ea_features =
    ea_detach_during_warmup and ea_started and not ea_joint
```

When detach_ea_features is true:

EaHead receives h_ts.detach()

That trains the Ea head but blocks Ea gradients from changing the EGNN/backbone.

11.9 Risk Ea loss

Only in joint mode:

```
risk_sample = I(runtime_risk_penalty > 0)
risk_weight = risk_scale * risk_sample
L_risk_ea =
    sum_b ea_abs_b * risk_weight_b / max(sum_b risk_weight_b, 1)
```

Added with:

```
risk_ea_loss_weight = 0.5
```

11.10 Total training loss

Base loss:

```
L_base =
    L_geom
    + geom_coarse_weight * L_geom_coarse
    + 0.2 * L_pinn
    + triangle_loss_weight * L_triangle
```

```
geom_coarse_weight = 0.5
triangle_loss_weight = 0.05
```

Full loss:

```
L_total = L_base

if risk pairs exist:
    L_total += risk_geom_loss_weight * L_risk_geom

if Ea head exists and ea_w > 0:
    L_total += ea_w * L_ea_weighted

if ea_joint and risk samples exist:
    L_total += risk_ea_loss_weight * L_risk_ea
```

12. Optimizer, Scheduler, Checkpointing

12.1 Optimizer

AdamW uses separate parameter groups:

base parameters:

```
lr = 1.5e-4
weight_decay = 1e-2
```

Ea head parameters:

```
lr = 3e-4
weight_decay = 1e-3
```

Gradient clipping:

```
clip_grad_norm_(model.parameters(), 1.0)
```

Mixed precision:

```
use_amp = config["amp"] and device.type == "cuda"
```

12.2 Scheduler

The active scheduler is CosineAnnealingWarmup.

In train_pipeline, scheduler warmup is hard-coded:

```
warmup_epochs = 100
min_lr = 1e-6
total_epochs = config["epochs"]
```

This is separate from the unused config key warmup_epochs = 40.

Learning rate rule:

```
if last_epoch < warmup_epochs:
    lr = base_lr * (last_epoch + 1) / warmup_epochs
else:
    progress = (last_epoch - warmup_epochs) / (total_epochs - warmup_epochs)
    cosine = 0.5 * (1 + cos(pi * progress))
    lr = min_lr + (base_lr - min_lr) * cosine
```

12.3 Validation selection

Before joint Ea training:

```
val_select = val_geom
```

After warmup:

```
val_select = val_geom + ea_select_weight * val_ea_norm
ea_select_weight = 0.5
```

val_ea_norm is the unweighted mean SmoothL1 Ea loss tracked as ea_norm.

At epoch:

```
epoch == ea_warmup_epochs + 1
```

the code resets:

```
best_val_loss = infinity
patience_counter = 0
```

12.4 Checkpoints

Every epoch saves:

```
psi_latest.pt
```

including:

```
model_state_dict
optimizer_state_dict
scheduler_state_dict
epoch
best_val_loss
patience_counter
```

```
metadata
history
```

On improvement:

```
psi_best.pt
```

At final evaluation:

```
psi_final.pt
psi_best.pt
detailed_analysis.json
training_history.json
psi_results_dashboard.html
```

13. Physics Ea Baseline

The physics Ea calculator is not the primary model output when the learned Ea head exists. It is fitted and reported as a baseline, and prediction uses it as a fallback for legacy checkpoints.

13.1 Bond force constants

Known element-pair constants are stored in BOND_FORCE_CONSTANTS.

Fallback for unknown pairs:

```
k_ij = 500 / max(r_cov_i + r_cov_j, 0.5)^3
```

Units are approximate kcal/(mol*Angstrom^2).

13.2 Reorganization energy

The bond set is the union of bonds in reactant, TS, and product geometries:

```
B_union = B_R union B_TS union B_P
```

For each bond:

```
lambda += 0.5 * k_ij * [
    (D_TS_ij - D_R_ij)^2
    + (D_P_ij - D_TS_ij)^2
]
```

13.3 Hammond index

Over active bonded pairs:

```
active if:
    pair is bonded in R or TS or P
    and abs(D_R_ij - D_P_ij) > 0.15
```

Sums:

```
sum_r += abs(D_TS_ij - D_R_ij)
sum_p += abs(D_TS_ij - D_P_ij)
```

Index:

```
if no active pairs or sum_r + sum_p < 1e-8:
    eta = 0.5
else:
    eta = sum_r / (sum_r + sum_p)
```

13.4 Marcus feature

Signed reaction energy:

```
DeltaE = e_p - e_r
```

Marcus Ea:

```
if lambda > 1e-6:
    Ea_marcus = (lambda / 4) * (1 + DeltaE / lambda)^2
```

```
else:
    Ea_marcus = 0.5 * abs(DeltaE)
```

13.5 OLS baseline

Feature vector:

```
x = [Ea_marcus, eta, DeltaE, 1]
```

Fit on training samples:

```
beta = argmin_beta ||X beta - y||_2
```

Implementation:

```
np.linalg.lstsq(X, y, rcond=None)
```

Physics prediction:

```
Ea_physics = x dot beta
```

14. Evaluation Flow

After training, the best checkpoint is loaded.

For each sample:

```
D_pred, D_coarse, Ea_pred_norm = model(...)
Ea_pred_kcal = Ea_pred_norm * ea_std + ea_mean
```

Geometry metrics are computed on the raw neural predicted distance matrix:

```
dist_MAE = sum_ij |D_pred_ij - D_TS_ij| * geom_mask_ij / max(sum_ij geom_mask_ij, 1)
dist_MAE_all = mean_ij |D_pred_ij - D_TS_ij|
```

Then, for the physics baseline only, the predicted distance matrix is post-processed before MDS:

```
D_post = max((D_pred + D_pred^T) / 2, 0)
diag(D_post) = 0
D_post = clamp_steric_collisions(D_post)
coords_pred = mds_aligned(D_post, reference_coords = midpoint aligned coords)
```

The physics OLS baseline is fitted on training samples using these predicted coordinates, then evaluated on all samples.

Final per-record output:

```
{
  rxn_id,
  split,
  n_atoms,
  dist_MAE,
  dist_MAE_all,
  D_I,
  D_pred,
  D_true,
  geom_mask,
  atom_types,
  Ea_true,
  Ea_pred,
  Ea_error,
  Ea_pred_physics,
  Ea_error_physics
}
```

Primary reported Ea:

```
Ea_pred = learned neural Ea if available
Ea_pred = physics Ea only as fallback
```

15. Prediction Flow

Prediction input:

```
reactant .log
product .log
psi_final.pt
```

Validation:

```
same atom count
same atom order and atom types
n_atoms <= max_atoms
all atom types exist in training atom_vocab
```

Features are built exactly like training:

```
D_R, D_P, D_I
atom ids
atom physical descriptors normalized by checkpoint metadata
de_rxn_norm
energy_feats_norm
```

Model output:

```
D_pred, _, Ea_pred_norm = model(...)
Ea_neural = Ea_pred_norm * ea_std + ea_mean
```

Prediction post-processing:

```
D_pred = max((D_pred + D_pred^T) / 2, 0)
diag(D_pred) = 0
D_pred = clamp_steric_collisions(D_pred)
D_pred = apply_spectator_constraints(D_pred, D_R, D_P, threshold=0.15, tol=0.05)
D_pred = enforce_triangle_inequality(D_pred)
validate_ts_geometry(D_pred)
coords_pred = mds_aligned(D_pred, reference_coords = aligned midpoint coords)
```

Steric floor:

```
min_distance_ij = 0.75 * (r_cov_i + r_cov_j)
D_pred_ij = max(D_pred_ij, min_distance_ij)
```

Spectator clamp:

```
if abs(D_R_ij - D_P_ij) <= 0.15:
    d_ref = (D_R_ij + D_P_ij) / 2
    lo = d_ref * (1 - 0.05)
    hi = d_ref * (1 + 0.05)
    D_pred_ij = clip(D_pred_ij, lo, hi)
```

Triangle post-processing:

```
if D_ij > D_ik + D_kj + 0.05:
    D_ij = D_ji = D_ik + D_kj
```

Prediction output:

```
Ea_pred = Ea_neural if available else Ea_physics
Ea_source = "neural" or "physics (no learned head in checkpoint)"
coords_pred = predicted TS coordinates
D_pred = post-processed predicted distance matrix
```

16. Dashboard Metrics

Energy metrics:

```
err_i = pred_i - true_i
MAE = mean_i |err_i|
```

```

RMSE = sqrt(mean_i err_i^2)
R2 = 1 - sum_i (true_i - pred_i)^2 / sum_i (true_i - mean(true))^2
Pearson = corrccoef(true, pred)
MAPE = mean_i |err_i| / |true_i| * 100

```

Geometry metrics use upper-triangle masked pairs:

```

MAE = mean_selected |D_pred - D_true|
RMSE = sqrt(mean_selected (D_pred - D_true)^2)
guess_MAE = mean_selected |D_I - D_true|
improve_pct = (1 - MAE / guess_MAE) * 100

```

17. Exact Implementation Differences

The two pipeline files currently differ in ways that matter for exact documentation.

17.1 Dataset extraction

psi_full_pipeline.py can parse b97d3.tar.gz.

psi_cloud_pipeline.py requires extracted_dataset.json to already exist.

17.2 Tail weighting default

psi_full_pipeline.py:

```
ea_tail_weighting_enabled = True
```

psi_cloud_pipeline.py:

```
ea_tail_weighting_enabled = False
```

The phase1_warm_start/train.log header does not show the high-Ea weighting startup line, consistent with the cloud default being false.

17.3 Coordinate noise

psi_full_pipeline.py has train-time coordinate noise:

```

coord_noise = 0.05
if is_train:
    X_R[:n] += Normal(0, 0.05)
    X_P[:n] += Normal(0, 0.05)

```

psi_cloud_pipeline.py does not apply this augmentation.

17.4 Main geometry weighting

psi_full_pipeline.py uses inverse-distance weighted geometry Huber:

```
w_ij = 1 / (D_TS_ij * M_ij + 1)
```

psi_cloud_pipeline.py uses the unweighted masked geometry Huber.

17.5 Risk feature backfill

psi_full_pipeline.py has ensure_sample_risk_features(...) in ReactionDataset.__getitem__, so older samples missing risk fields can be patched at access time.

psi_cloud_pipeline.py assumes the risk fields already exist in each sample.

17.6 Parser note

PLANNING.md states that parse_log_content handles both Standard Nuclear Orientation and Coordinates (Angstroms). The active code in both Python files currently only matches Standard Nuclear Orientation for coordinates.

18. Training Algorithm Pseudocode

```

function train_pipeline(config):
    device = resolve_device(config)
    extract_raw_data(config)
    samples, atom_vocab, atom_types_map = build_reaction_samples(config)
    train_indices, val_indices = make_train_val_split(samples, config)

```

```

stats = compute_normalization(samples, train_indices)

train_dataset = ReactionDataset(samples, stats, is_train=True in full pipeline)
eval_dataset = ReactionDataset(samples, stats, is_train=False)
train_loader = DataLoader(train_indices, shuffle=True)
val_loader = DataLoader(val_indices, shuffle=False)

model = PSI(config, num_atom_types)
optimizer = AdamW(base_params, Ea-head params)
scheduler = CosineAnnealingWarmup(warmup_epochs=100)

if psi_latest.pt exists:
    resume model, optimizer, scheduler, epoch, history

for epoch in start_epoch..epochs:
    train_metrics = run_epoch(train_loader, is_train=True)
    val_metrics = run_epoch(val_loader, is_train=False)

    if epoch == ea_warmup_epochs + 1:
        reset best_val_loss and patience_counter

    if epoch <= ea_warmup_epochs:
        val_select = val_geom
    else:
        val_select = val_geom + ea_select_weight * val_ea_norm

    scheduler.step()
    append metrics to history

    if val_select improves:
        save psi_best.pt
        patience_counter = 0
    else:
        patience_counter += 1

    save psi_latest.pt

    if patience_counter >= patience:
        break

save training_history.json
load psi_best.pt
evaluate all samples
fit physics Ea baseline on train predictions
save detailed_analysis.json, psi_final.pt, dashboard

```

19. Forward-Pass Algorithm Pseudocode

```

function PSI.forward(D_R, D_I, D_P, mask, atom_ids, atom_phys, de_rxn, energy_feats):
    h_core = PSICore(D_R, D_I, D_P, mask, atom_ids, atom_phys)
    atom_emb = embedding(atom_ids)

    D_coarse = GeometryHead(
        h_core, atom_emb, atom_phys, D_R, D_I, D_P, mask
    )

    if EGNN enabled:
        node_feats = concat(atom_emb, atom_phys)
        x_init = torch_mds_coords(detach(D_coarse), mask)
        h_ts, x_ts = EGNN(node_feats, x_init, mask)
        D_pred = pairwise_distance(x_ts)

```

```

    if de_rxn is provided:
        if detach_ea_features:
            h_for_ea = detach(h_ts)
        else:
            h_for_ea = h_ts
        Ea_pred_norm = EaHead(h_for_ea, mask, de_rxn, energy_feats)
    else:
        Ea_pred_norm = None
else:
    D_pred = D_coarse
    Ea_pred_norm = None

return D_pred, D_coarse, Ea_pred_norm

```

20. Loss Algorithm Pseudocode

```

function run_epoch(..., epoch, is_train):
    ea_started = epoch >= ea_loss_start_epoch
    ea_joint = epoch > ea_warmup_epochs

    if ea_joint:
        ea_w = ea_loss_weight
    else if ea_started:
        ea_w = ea_warmup_loss_weight
    else:
        ea_w = 0

    detach_ea_features = ea_detach_during_warmup and ea_started and not ea_joint

    for batch in loader:
        D_pred, D_coarse, Ea_pred_norm = model(..., detach_ea_features)

        compute L_geom
        compute L_geom_coarse
        compute L_pinn
        compute L_triangle

        loss = L_geom
            + 0.5 * L_geom_coarse
            + 0.2 * L_pinn
            + 0.05 * L_triangle

        if risk pairs exist:
            loss += 0.2 * L_risk_geom

        if Ea_pred_norm exists:
            y_ea = (Ea_raw - ea_mean) / ea_std
            ea_abs = SmoothL1(Ea_pred_norm - y_ea)
            L_ea_weighted = weighted_mean(ea_abs, ea_tail_weight)

            if ea_w > 0:
                loss += ea_w * L_ea_weighted

            if ea_joint and risk samples exist:
                loss += 0.5 * L_risk_ea

    if is_train:
        skip batch if loss is non-finite
        backprop with GradScaler if AMP is enabled
        clip gradients to 1.0
        optimizer.step()

```